

Министерство образования Саратовской области
Государственное автономное профессиональное образовательное
учреждение Саратовской области
«Новоузенский агротехнологический техникум»

КУРСОВАЯ РАБОТА

по МДК 01.01 «Разработка программных модулей»
на тему: «Разработка программного модуля учета туров и заявок
туристического агентства»

Выполнил: студент(ка) группы _____

Руководитель: преподаватель Мажилин А.А.

Новоузенск 2026

Задание на курсовую работу

Таблица 0.1 – Задание на разработку

Показатель	Содержание
Тема	Разработка программного модуля учета туров и заявок туристического агентства
Цель	Разработать настольный программный модуль для подбора туров, ведения клиентов, оформления заявок, учета оплат и формирования договора.
Исходные данные	Предметная область туристического агентства; среда разработки .NET 8; фреймворк Avalonia UI; СУБД PostgreSQL; библиотека Npgsql.
Результат	Работающее приложение TourAgency1, структура базы данных, тестовые данные, пояснительная записка и руководство пользователя.

В ходе выполнения работы необходимо проанализировать предметную область, определить требования к программному модулю, спроектировать структуру данных и пользовательский интерфейс, реализовать программный код, выполнить проверку работоспособности и подготовить пользовательскую инструкцию.

Таблица 0.2 – Календарный план выполнения работы

№	Этап	Срок	Результат
1	Выбор темы и анализ предметной области	1 неделя	Определены цель, объект, предмет и задачи
2	Формирование требований	1 неделя	Составлен перечень функций и ограничений
3	Проектирование базы данных и интерфейса	1 неделя	Подготовлена структура таблиц и экранных форм
4	Разработка программного модуля	2 недели	Реализованы модели, репозиторий, ViewModel и интерфейс
5	Тестирование и исправление ошибок	1 неделя	Проверены основные пользовательские сценарии
6	Оформление пояснительной записки	1 неделя	Подготовлен итоговый документ и приложения

Содержание

Введение.....	4
1 Постановка задачи.....	5
1.1 Анализ предметной области.....	5
1.2 Функциональные и нефункциональные требования.....	6
1.3 Выбор средств разработки.....	7
2 Проектирование программного модуля.....	9
2.1 Архитектура приложения.....	9
2.2 Проектирование базы данных.....	10
2.3 Проектирование пользовательского интерфейса.....	12
3 Разработка программного модуля.....	13
3.1 Реализация модели данных.....	13
3.2 Реализация доступа к данным.....	14
3.3 Реализация логики интерфейса.....	15
3.4 Дополнительные проектные материалы.....	16
4 Тестирование и руководство пользователя.....	21
4.1 Методика тестирования.....	21
4.2 Результаты тестирования.....	22
4.3 Руководство пользователя.....	23
4.4 Программа испытаний.....	24
Заключение.....	27
Список использованных источников.....	28
Приложения.....	29

ВВЕДЕНИЕ

Туристическое агентство работает с большим количеством взаимосвязанных данных: направлениями, отелями, турами, клиентами, заявками и оплатами. При ручной обработке этих данных возрастает риск ошибок при расчете стоимости, контроле свободных мест и подготовке документов для клиента. Поэтому разработка специализированного программного модуля учета туров является актуальной учебно-практической задачей.

Объектом разработки является процесс обслуживания клиентов туристического агентства. Предметом разработки выступает программный модуль, обеспечивающий хранение, поиск и обработку данных о турах, клиентах, заявках и платежах.

Цель курсовой работы – разработать настольное приложение для туристического агентства, позволяющее сотруднику подбирать туры по параметрам, регистрировать клиентов, создавать заявки, фиксировать оплаты и формировать текст договора.

Для достижения цели необходимо решить следующие задачи: изучить предметную область; сформулировать требования к модулю; выбрать технологии реализации; спроектировать структуру базы данных; реализовать пользовательский интерфейс; организовать доступ к PostgreSQL; выполнить тестирование основных сценариев; подготовить руководство пользователя.

Практическая значимость работы состоит в том, что созданный модуль может использоваться как учебный прототип автоматизированного рабочего места сотрудника туристического агентства. Приложение демонстрирует полный цикл работы с данными: ввод, проверку, хранение, поиск, изменение связанных записей и подготовку печатного документа.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Анализ предметной области

Туристическое агентство оказывает услуги по подбору и оформлению туристических продуктов. Сотрудник агентства должен быстро найти подходящий тур, проверить наличие свободных мест, оформить заявку на клиента, рассчитать итоговую стоимость поездки, принять оплату и подготовить договор.

Основными информационными объектами предметной области являются клиент, направление, отель, тур, заявка и оплата. Клиент содержит контактные и паспортные данные. Направление описывает страну, город и название туристического маршрута. Отель связан с направлением и имеет категорию, адрес и описание. Тур хранит цену, даты поездки, тип питания, вид транспорта и количество свободных мест. Заявка связывает клиента с выбранным туром, а оплата отражает финансовые операции по заявке.

Без автоматизации сотруднику приходится вести несколько журналов или таблиц. Это неудобно при поиске туров по нескольким условиям и опасно при одновременном изменении остатков мест. В разработанном модуле эти действия объединены в одном пользовательском сценарии.

Типовой рабочий день сотрудника туристического агентства включает консультацию клиента, подбор вариантов поездки, уточнение дат и бюджета, проверку актуальности предложения, регистрацию заявки и прием предоплаты. Если эти сведения хранятся разрозненно, сотрудник тратит время на повторный ввод информации и может допустить ошибку при расчете полной стоимости.

Особенность рассматриваемой предметной области состоит в высокой связности данных. Тур нельзя рассматривать отдельно от направления и отеля, а оплату нельзя корректно обработать без заявки. Поэтому при проектировании модуля важно не только отобразить таблицы, но и описать правила перехода данных из одного состояния в другое.

Разрабатываемое приложение ориентировано на сотрудника, который работает за настольным компьютером и выполняет повторяющиеся операции. Для такого сценария подходит табличный интерфейс: он позволяет быстро сравнивать туры, сортировать строки визуально, выбирать записи и видеть финансовое состояние заявок.

Таблица 1.1 – Информационные объекты предметной области

Объект	Данные	Назначение
Клиенты	ФИО, телефон, email, паспортные данные	Регистрация и выбор клиента при оформлении заявки
Направления	Страна, город, название, описание	Поиск и группировка туристических продуктов
Отели	Название, категория, адрес, описание	Описание условий размещения
Туры	Даты, цена, питание, транспорт, свободные места	Основной объект подбора и продажи
Заявки	Клиент, тур, количество туристов, стоимость, статус	Фиксация бронирования
Оплаты	Заявка, сумма, способ, дата, комментарий	Учет платежей и обновление статуса

1.2 Функциональные и нефункциональные требования

Таблица 1.2 – Функциональные требования

Код	Требование	Приоритет
F1	Загрузка справочников, туров, клиентов, заявок и оплат из базы данных	Обязательное
F2	Поиск туров по направлению, дате начала, максимальной цене и количеству туристов	Обязательное
F3	Добавление нового клиента с проверкой ФИО и телефона	Обязательное
F4	Создание заявки с автоматическим расчетом итоговой стоимости	Обязательное
F5	Уменьшение количества свободных мест после оформления заявки	Обязательное
F6	Добавление оплаты и изменение статуса заявки	Обязательное
F7	Формирование текста договора и сохранение HTML-файла для печати	Желательное

Нефункциональные требования определяют качество реализации. Приложение должно иметь единообразный интерфейс, корректно сообщать об ошибках, не допускать отрицательного количества туристов, использовать параметризованные SQL-запросы и хранить данные в реляционной базе.

Для надежности операций оформления заявки и оплаты используется транзакционная обработка. Это позволяет согласованно создавать заявку, изменять остаток мест и обновлять статус оплаты.

Пользовательские сообщения должны быть короткими и понятными. При ошибке ввода программа сообщает, какое поле заполнено неправильно, а при успешной операции выводит подтверждение действия. Это снижает вероятность повторного выполнения одной и той же операции.

Модуль должен быть пригоден для дальнейшего расширения. Например, в будущем можно добавить редактирование справочников, отмену заявки, экспорт отчетов или авторизацию сотрудников. Поэтому проект разделен на модели, представления и слой доступа к данным.

Таблица 1.3 – Ограничения предметной области

Показатель	Правило	Реализация
Количество туристов	Не меньше 1	Проверяется в ViewModel и базе данных
Цена тура	Неотрицательное значение	Ограничение CHECK в таблице tours
Сумма оплаты	Больше 0	Проверяется перед записью платежа
Категория отеля	От 1 до 5 звезд	Ограничение CHECK в таблице hotels
Даты тура	Дата окончания не раньше даты начала	Ограничение tours_dates_check
Статус заявки	new, confirmed, paid, cancelled	Ограничение bookings_status_check
Телефон клиента	Уникальное значение	Индекс ux_clients_phone

1.3 Выбор средств разработки

Таблица 1.3 – Используемые средства разработки

Средство	Назначение	Обоснование
.NET 8	Платформа выполнения приложения	Поддерживает современный C#, асинхронность и настольные приложения
C#	Язык программирования	Строгая типизация, развитая стандартная библиотека, удобная работа с классами
Avalonia UI	Фреймворк интерфейса	Кроссплатформенный XAML-интерфейс и DataGrid для табличных данных
ReactiveUI	Связь интерфейса и логики	Команды, уведомления об изменениях свойств, MVVM-подход
PostgreSQL	Система управления	Надежная реляционная СУБД с

	базой данных	ограничениями целостности
Npgsql	Драйвер доступа к PostgreSQL	Асинхронные запросы, параметры SQL, транзакции

Выбор Avalonia UI обусловлен тем, что фреймворк позволяет описывать интерфейс декларативно в XAML и отделять визуальную часть от логики. Это удобно для учебного проекта, где важно показать структуру программного модуля и связи между слоями.

PostgreSQL выбран как промышленная СУБД, поддерживающая внешние ключи, транзакции, индексы и ограничения. Для рассматриваемой задачи это важно, потому что данные о турах и заявках должны оставаться согласованными даже при изменении нескольких таблиц в рамках одного действия.

Использование .NET 8 позволяет применять современный синтаксис C#, асинхронные методы и пакетную систему NuGet. Проект содержит явные зависимости в файле TourAgency1.csproj, поэтому его можно восстановить и собрать стандартными средствами SDK.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО МОДУЛЯ

2.1 Архитектура приложения

Приложение построено по схеме, близкой к MVVM. Отдельные классы моделей описывают данные предметной области. Представления Avalonia отвечают за отображение и ввод данных. Класс ToursViewModel содержит состояние экрана, команды и проверку пользовательских действий. Класс TourRepository инкапсулирует SQL-запросы и операции с PostgreSQL.

Такое разделение повышает сопровождаемость программы. Изменение структуры интерфейса не требует переписывания SQL-запросов, а изменение способа хранения данных не затрагивает XAML-разметку. Логика сценариев сосредоточена во ViewModel, что соответствует требованиям к структурированности программного модуля.

Таблица 2.1 – Структура проекта TourAgency1

Слой	Файлы	Ответственность
Views	MainWindow.axaml, ToursView.axaml	Отображение главного окна, вкладок, таблиц, полей ввода и кнопок
ViewModels	MainViewModel, ToursViewModel	Состояние интерфейса, команды загрузки, поиска, добавления клиента, создания заявки, оплаты и договора
Models	Tour, Client, Booking, Payment, Hotel, Destination, AgencyData	Типизированное представление данных
Data	TourRepository, DatabaseSettings	Подключение к PostgreSQL и выполнение SQL-запросов
Database	01_create_database.sql, 02_create_tables.sql, 03_seed_data.sql	Создание и первичное заполнение учебной базы

Текстовая схема взаимодействия компонентов имеет следующий вид: пользователь выполняет действие в ToursView; команда передается в ToursViewModel; ViewModel проверяет входные данные и обращается к TourRepository; репозиторий выполняет запрос к PostgreSQL; результат возвращается в модели и отображается в таблицах интерфейса.

Жизненный цикл данных при создании заявки состоит из нескольких шагов. Сначала пользователь выбирает тур и клиента в интерфейсе. Затем

ViewModel проверяет, что оба объекта выбраны, и передает в репозиторий идентификаторы тура и клиента, количество туристов и примечание. Репозиторий в транзакции получает цену и остаток мест, рассчитывает полную стоимость, записывает заявку и уменьшает остаток.

Жизненный цикл данных при оплате устроен аналогично. Пользователь выбирает заявку, указывает сумму и способ оплаты. После проверки суммы репозиторий добавляет запись в payments и пересчитывает сумму всех платежей по заявке. Если общая оплата равна стоимости или превышает ее, заявка получает статус paid.

Таблица 2.2 – Поток данных при поиске тура

Шаг	Операция	Компонент	Данные
1	Пользователь вводит фильтры	ToursView	DestinationFilter, StartFromText, MaxPriceFilter
2	Команда поиска запускает метод SearchToursAsync	ToursViewModel	Проверка даты, цены и количества туристов
3	Репозиторий выполняет SQL-запрос	TourRepository	Параметризованный SELECT с JOIN
4	Результат преобразуется в список Tour	Models	Объекты Tour
5	Интерфейс обновляет таблицу	ToursView	DataGrid с найденными турами

2.2 Проектирование базы данных

Таблица 2.2 – Таблицы базы данных

Таблица	Ключ	Основные поля	Назначение
clients	id	full_name, phone, email, passport_number, created_at	Клиенты агентства
destinations	id	country, city, name, description	Направления отдыха
hotels	id	destination_id, name, stars, address, description	Отели, связанные с направлениями
tours	id	destination_id, hotel_id, title, price_per_person, start_date, end_date, available_spots	Туристические продукты
bookings	id	tour_id, client_id, people_count, total_price, status, booked_at, notes	Заявки клиентов
payments	id	booking_id, paid_at, amount, method, comment	Оплаты по заявкам

Связи базы данных обеспечиваются внешними ключами. Отели и туры связаны с направлениями, заявки связаны с турами и клиентами, оплаты связаны с заявками. Для платежей используется каскадное удаление при удалении заявки, поскольку платеж без заявки теряет смысл.

Целостность данных дополнительно поддерживается ограничениями CHECK. Категория отеля ограничена диапазоном от 1 до 5, стоимость тура и заявки не может быть отрицательной, количество мест и туристов контролируется числовыми ограничениями, а статус заявки ограничен набором значений new, confirmed, paid и cancelled.

Индексы созданы для полей, которые часто используются в связях и фильтрах. Например, индексы по destination_id, hotel_id, start_date и price_per_person ускоряют выборку туров. Индексы по booking_id в таблице payments и по client_id в таблице bookings ускоряют получение связанных записей.

Учебная база наполняется тестовыми данными с помощью файла 03_seed_data.sql. В нем создаются клиенты, направления, отели, туры, заявки и платежи. Наличие начальных данных позволяет проверить программу сразу после создания таблиц, не вводя все сведения вручную.

Таблица 2.4 – Словарь ключевых полей базы данных

Поле	Тип	Назначение	Ограничение
clients.full_name	varchar(200)	ФИО клиента	Обязательное
clients.phone	varchar(30)	Телефон клиента	Обязательное, уникальное
tours.price_per_person	numeric(12,2)	Цена за одного туриста	Неотрицательная
tours.available_spots	integer	Количество доступных мест	Неотрицательное
bookings.people_count	integer	Количество туристов в заявке	Больше 0
bookings.total_price	numeric(12,2)	Итоговая стоимость заявки	Рассчитывается автоматически
bookings.status	varchar(30)	Состояние заявки	Ограниченный перечень
payments.amount	numeric(12,2)	Сумма платежа	Больше 0

2.3 Проектирование пользовательского интерфейса

Главное окно имеет заголовок «Туристическое агентство» и содержит рабочую область с вкладками. Верхняя панель используется для быстрого подбора путевок по направлению, дате начала, максимальной цене и количеству туристов. Ниже выводится статусное сообщение, позволяющее пользователю понять результат выполненной операции.

Вкладка «Путевки» содержит таблицу туров и форму создания заявки. Вкладка «Клиенты» содержит список клиентов и форму регистрации нового клиента. Вкладка «Заявки и оплаты» объединяет таблицу заявок, форму добавления платежа и таблицу оплат. Вкладка «Договор» показывает текст договора и команды формирования и сохранения. Вкладка «Справочники» выводит направления и отели.

Интерфейс проектировался с учетом повторяемости операций. Наиболее частое действие, подбор путевки, вынесено в верхнюю часть экрана и доступно независимо от выбранной вкладки. Табличные данные занимают основную площадь окна, потому что сотруднику важно видеть несколько записей одновременно.

Для визуальной единообразности в XAML заданы общие стили для TextBox, ComboBox, Button, TabControl, TabItem и DataGrid. Основная кнопка выделяется классом primary, а справочные подписи используют стиль label. Это делает интерфейс спокойным и удобным для длительной работы.

Таблица 2.3 – Основные элементы управления

Элемент	Назначение
Подобрать	Поиск туров по заданным фильтрам
Сбросить	Очистка фильтров и повторная загрузка данных
Оформить заявку	Создание заявки для выбранного клиента и тура
Добавить клиента	Регистрация нового клиента
Добавить оплату	Фиксация платежа по выбранной заявке
Сформировать	Подготовка текста договора
Сохранить и печатать	Создание HTML-файла договора и открытие печати

3 РАЗРАБОТКА ПРОГРАММНОГО МОДУЛЯ

3.1 Реализация модели данных

Модели реализованы как классы C# с открытыми свойствами. Такой подход удобен для привязки данных в Avalonia и для чтения результатов SQL-запросов. В проекте выделены классы Tour, Client, Booking, Payment, Hotel, Destination и AgencyData.

Класс Tour содержит вычисляемое свойство Nights, которое определяет количество ночей на основе дат начала и окончания тура. Класс Booking содержит вычисляемое свойство Balance, показывающее остаток к оплате. Это уменьшает дублирование расчетов в интерфейсе.

Все строковые свойства моделей инициализируются пустой строкой. Это снижает риск ошибок, связанных с null-значениями, и упрощает отображение данных в таблицах. Для дат, получаемых из PostgreSQL, в репозитории применяется DateTime.SpecifyKind с DateTimeKind.Unspecified, поскольку даты тура и локальные даты операций не требуют преобразования часовых поясов.

Модель AgencyData используется как контейнер для пакетной загрузки данных. Вместо нескольких независимых возвратов метод GetAgencyDataAsync возвращает один объект, содержащий списки направлений, отелей, туров, клиентов, заявок и платежей. Это упрощает обновление интерфейса после операций изменения данных.

Таблица 3.1 – Основные классы моделей

Класс	Назначение	Ключевые свойства
Tour	Туристический продукт	Nights, DisplayName
Client	Клиент агентства	FullName, Phone, Email, PassportNumber
Booking	Заявка	TotalPrice, PaidAmount, Balance, Status
Payment	Платеж	Amount, Method, PaidAt
AgencyData	Пакет загруженных данных	Списки справочников и операций

3.2 Реализация доступа к данным

Доступ к данным сосредоточен в классе TourRepository. Репозиторий получает строку подключения из DatabaseSettings. По умолчанию

приложение подключается к базе `tour_agency`, но строка подключения может быть переопределена переменной окружения `TOUR_AGENCY_CONNECTION_STRING`.

Для выполнения SQL-запросов используется `NpgsqlDataSource`. Все операции чтения и изменения данных реализованы асинхронно, что предотвращает блокировку пользовательского интерфейса во время работы с базой данных.

Поиск туров выполняется параметризованным запросом. Параметры `destination`, `start_from`, `max_price` и `people_count` не подставляются в SQL-строку напрямую, что снижает риск SQL-инъекций и ошибок форматирования.

Метод `GetAgencyDataAsync` загружает все данные, необходимые для начального состояния интерфейса. После добавления клиента, создания заявки или добавления оплаты этот метод вызывается повторно. Такой подход проще локального изменения нескольких коллекций и гарантирует, что интерфейс показывает актуальное состояние базы.

Для преобразования пустых строк в значения базы данных используется вспомогательный метод `ToDbText`. Если пользователь оставляет необязательное поле пустым, в базу передается `DBNull.Value`. Это позволяет отличать неизвестное значение от осмысленно заполненной строки.

```
public async Task<IReadOnlyList<Tour>> SearchToursAsync(
    string? destination, DateTime? startFrom, decimal? maxPrice, int peopleCount)
{
    const string sql = """
        SELECT t.id, t.destination_id, t.hotel_id, d.name, d.country, d.city,
               h.name, h.stars, t.title, t.description, t.price_per_person,
               t.start_date, t.end_date, t.available_spots, t.meal_type,
        t.transport_type
        FROM tours t
        JOIN destinations d ON d.id = t.destination_id
        JOIN hotels h ON h.id = t.hotel_id
        WHERE (@destination IS NULL OR d.name ILIKE '%' || @destination || '%')
            AND (@start_from IS NULL OR t.start_date >= @start_from)
            AND (@max_price IS NULL OR t.price_per_person <= @max_price)
            AND t.available_spots >= @people_count
        ORDER BY t.start_date, t.price_per_person, t.id;
        """;
}
```

Оформление заявки является критической операцией, потому что одновременно создается запись в таблице bookings и уменьшается количество свободных мест в таблице tours. В реализации применяется транзакция, а выбранный тур читается с блокировкой FOR UPDATE. Это защищает остаток мест от некорректного изменения при параллельной работе.

Добавление оплаты также выполняется в транзакции. После вставки платежа запрос пересчитывает сумму оплат по заявке и устанавливает статус paid, если оплаченная сумма покрывает полную стоимость. Если заявка была новой и по ней внесена частичная оплата, статус меняется на confirmed.

Применение транзакций особенно важно в учебном проекте, потому что показывает правильный способ реализации бизнес-операций, затрагивающих несколько таблиц. Без транзакции могла бы возникнуть ситуация, когда заявка создана, но количество свободных мест не изменилось, или платеж добавлен, но статус заявки не обновлен.

SQL-запросы в репозитории сгруппированы по смыслу. Отдельные методы GetDestinationsAsync, GetHotelsAsync, GetToursAsync, GetClientsAsync, GetBookingsAsync и GetPaymentsAsync отвечают за чтение конкретных сущностей. Это упрощает проверку и дальнейшее изменение отдельных выборок.

3.3 Реализация логики интерфейса

ToursViewModel наследуется от ReactiveObject и использует RaiseAndSetIfChanged для уведомления интерфейса об изменениях. Табличные данные хранятся в AvaloniaList, что удобно для привязки к DataGrid.

Команды LoadDataCommand, SearchToursCommand, ResetFiltersCommand, AddClientCommand, CreateBookingCommand, AddPaymentCommand, GenerateContractCommand и SaveContractCommand связывают кнопки интерфейса с методами ViewModel. Ошибки пользовательского ввода перехватываются и выводятся в строку состояния.

При выборе тура автоматически рассчитывается ожидаемая сумма оплаты на основе цены и количества туристов. При выборе заявки автоматически подставляется остаток к оплате и формируется текст договора. Такой подход сокращает количество ручных действий пользователя.

Проверка дат и числовых значений выполняется до обращения к базе. Дата начала тура принимается в форматах dd.ММ.уууу, d.М.уууу и уууу-ММ-dd, что делает ввод более гибким. Для денежных сумм используется попытка разбора как в текущей культуре, так и в инвариантной культуре.

Формирование договора реализовано как создание многострочного текста по данным выбранной заявки. Сохранение выполняется в HTML-файл с кодировкой UTF-8 и вызовом печати через браузер или системный просмотрщик. Такой вариант прост в реализации и подходит для учебного прототипа.

Таблица 3.2 – Методы пользовательской логики

Метод	Действие	Результат
LoadDataAsync	Загружает справочники и операции из БД	Сообщает количество записей
SearchToursAsync	Разбирает фильтры и вызывает SearchToursAsync репозитория	Показывает число найденных туров
AddClientAsync	Проверяет ФИО и телефон, создает клиента	Очищает поля формы
CreateBookingAsync	Проверяет выбранный тур и клиента, создает заявку	Выбирает созданную заявку
AddPaymentAsync	Проверяет сумму оплаты, добавляет платеж	Обновляет статус заявки
GenerateContract	Формирует текст договора	Использует данные выбранной заявки

3.4 Дополнительные проектные материалы

Для более полного описания разработки рассмотрим внутренние алгоритмы и правила обработки ошибок. Эти материалы дополняют раздел разработки и показывают, каким образом программный модуль выполняет основные операции не только с точки зрения пользователя, но и с точки зрения структуры кода.

Алгоритм поиска туров начинается с чтения значений фильтров из полей интерфейса. Если пользователь не указал направление, дату или максимальную цену, соответствующее условие не ограничивает выборку. Количество туристов всегда приводится к значению не меньше единицы, поэтому в запрос не попадает некорректное нулевое или отрицательное значение.

После подготовки фильтров ViewModel вызывает метод репозитория. SQL-запрос выбирает туры с присоединением направлений и отелей, потому что в таблице интерфейса должны отображаться не только идентификаторы, но и человекочитаемые названия. Результат сортируется по дате начала, цене и идентификатору, чтобы пользователь сначала видел ближайшие и более доступные варианты.

Алгоритм оформления заявки содержит несколько обязательных проверок. Программа убеждается, что выбран тур и клиент. Затем в транзакции получает цену и остаток мест. Если мест недостаточно, транзакция завершается без изменений. Если мест достаточно, программа рассчитывает полную стоимость, создает заявку и уменьшает количество доступных мест.

Алгоритм добавления оплаты проверяет выбранную заявку и положительность суммы. После вставки платежа выполняется запрос, который суммирует все оплаты по заявке. На основании суммы программа обновляет статус: paid для полностью оплаченной заявки и confirmed для частично оплаченной новой заявки.

Таблица 3.3 – Основные алгоритмы программного модуля

Алгоритм	Входные данные	Выходные данные	Реализация
Поиск туров	Направление, дата, цена, туристы	Список подходящих туров	SELECT с JOIN и параметрами
Создание заявки	Тур, клиент, количество туристов	Заявка и уменьшенный остаток мест	Транзакция, SELECT FOR UPDATE, INSERT, UPDATE
Добавление оплаты	Заявка, сумма, способ оплаты	Платеж и обновленный статус	Транзакция, INSERT, UPDATE с агрегированием

Формирование договора	Данные выбранной заявки	Текст договора	Строковый шаблон GenerateContract
-----------------------	-------------------------	----------------	-----------------------------------

Подготовка базы данных разделена на три сценария. Первый сценарий создает базу `tour_agency`. Второй сценарий пересоздает таблицы и индексы. Третий сценарий добавляет демонстрационные записи и корректирует значения последовательностей идентификаторов. Такое разделение удобно при повторной настройке учебного стенда.

При проектировании использована нормализованная структура. Информация об отеле не дублируется в каждой заявке, а хранится в отдельной таблице `hotels`. Информация о направлении вынесена в `destinations`. Это уменьшает объем повторяющихся данных и снижает риск расхождений, например разных написаний одного и того же города.

В заявке хранится рассчитанная стоимость `total_price`. Это сделано намеренно: цена тура может измениться в будущем, но уже созданная заявка должна сохранять стоимость, согласованную с клиентом на момент оформления. Остаток к оплате при этом рассчитывается как разность между `total_price` и суммой платежей.

Классы моделей не содержат SQL-запросов и не зависят от Avalonia. Это делает их простыми объектами предметной области. Такое решение соответствует принципу разделения ответственности: модель описывает данные, репозиторий занимается хранением, `ViewModel` отвечает за сценарии работы.

Вычисляемые свойства используются только там, где значение можно однозначно получить из других полей. Например, `Balance` не хранится в базе как отдельное поле, потому что его можно рассчитать из `total_price` и `paid_amount`. Это уменьшает вероятность противоречивых данных.

Таблица 3.4 – Словарь ключевых полей

Поле	Тип	Назначение	Ограничение
<code>clients.full_name</code>	<code>varchar(200)</code>	ФИО клиента	Обязательное поле
<code>clients.phone</code>	<code>varchar(30)</code>	Телефон клиента	Обязательное и

			уникальное поле
tours.price_per_person	numeric(12,2)	Цена за одного туриста	Неотрицательное значение
tours.available_spots	integer	Количество доступных мест	Неотрицательное значение
bookings.people_count	integer	Количество туристов в заявке	Больше 0
bookings.total_price	numeric(12,2)	Итоговая стоимость заявки	Рассчитывается автоматически
bookings.status	varchar(30)	Состояние заявки	new, confirmed, paid, cancelled
payments.amount	numeric(12,2)	Сумма платежа	Больше 0

В запросах чтения активно используются JOIN. Например, при получении заявок программа объединяет bookings, clients, tours, destinations, hotels и payments. Это позволяет сразу вывести имя клиента, название тура, направление, отель, полную стоимость, оплаченную сумму и статус.

Для получения оплаченной суммы используется агрегирование SUM по платежам. LEFT JOIN позволяет корректно показать заявку даже в том случае, если по ней еще нет оплат. В этом случае COALESCE заменяет отсутствующую сумму платежей на 0.

Для обработки ошибок используются исключения. Репозиторий сообщает о невозможности найти выбранный тур или о недостатке мест, а ViewModel перехватывает исключение и преобразует его в статусное сообщение для пользователя. Такой подход не скрывает проблему и не завершает приложение аварийно.

Особое внимание уделено параметрам SQL-команд. Вместо формирования строки через конкатенацию используются AddWithValue и параметры с типами NpgsqlDbType. Это повышает безопасность и надежность работы с датами, числами и текстовыми значениями.

Метод ResetFiltersAsync очищает все поля поиска и повторно загружает данные из базы. Это важная функция для пользователя, потому что после нескольких фильтров нужно быстро вернуться к полному списку туров без ручного удаления значений из каждого поля.

Метод LoadDataAsync вызывается при создании ViewModel. Благодаря этому приложение пытается загрузить данные сразу после запуска. Если подключение к базе недоступно, пользователь получает сообщение об ошибке, а окно приложения остается открытым.

Таблица 3.5 – Обработка исключительных ситуаций

Ситуация	Причина	Реакция программы
Нет подключения к PostgreSQL	Неверная строка подключения, сервер недоступен	Показать сообщение в строке состояния
Некорректная дата	Дата введена не в поддерживаемом формате	Сообщить о формате ДД.ММ.ГГГГ
Некорректная сумма	Введен текст или отрицательное значение	Отменить операцию и показать ошибку
Недостаточно мест	Количество туристов больше available_spots	Не создавать заявку
Пустые обязательные поля клиента	Не введены ФИО или телефон	Не добавлять клиента

4 ТЕСТИРОВАНИЕ И РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

4.1 Методика тестирования

Тестирование программного модуля выполнялось по основным пользовательским сценариям. Проверялись запуск приложения, загрузка данных из PostgreSQL, поиск туров, добавление клиента, оформление заявки, учет оплаты, формирование договора и обработка некорректного ввода.

Отдельно проверялась техническая сборка проекта командой `dotnet build TourAgency1.sln`. Сборка завершилась успешно, предупреждения и ошибки отсутствовали. Это подтверждает корректность зависимостей, синтаксиса C# и XAML-разметки на уровне компиляции.

Методика тестирования включает позитивные и негативные сценарии. Позитивные сценарии подтверждают, что пользователь может выполнить основную операцию при корректных данных. Негативные сценарии проверяют, что программа не выполняет опасное действие и сообщает причину отказа.

Для тестирования использовались учебные данные из файла `03_seed_data.sql`. В базе присутствуют несколько направлений, отелей, туров, клиентов, заявок и оплат. Это позволяет проверять фильтрацию, расчеты и статусы без предварительного ручного заполнения справочников.

Таблица 4.1 – Тестовые данные

Группа данных	Пример	Назначение
Клиенты	Иванов И.П., Петрова А.С., Смирнов О.А.	Проверка выбора клиента и добавления заявок
Направления	Сочи, Стамбул, Ереван, Калининград	Проверка поиска по стране, городу и названию
Тур	Черноморская неделя, Стамбул на двоих берегах, Вкус Армении	Проверка фильтрации и расчета стоимости
Заявки	confirmed, paid, new	Проверка отображения статусов и баланса
Оплаты	Предоплата и полная оплата	Проверка изменения статуса заявки

Таблица 4.1 – Тестовые сценарии

№	Проверка	Действия	Ожидаемый результат	Статус
T1	Сборка проекта	Выполнить <code>dotnet build</code>	Сборка успешна, 0 ошибок	Пройден

		TourAgency1.sln		
T2	Загрузка данных	Открыть приложение при доступной БД	Отображаются туры, клиенты, заявки и справочники	Пройден
T3	Поиск тура	Ввести направление «Сочи», дату и цену	Список содержит подходящие туры	Пройден
T4	Пустой поиск	Задать слишком низкую цену	Выводится сообщение об отсутствии путевок	Пройден
T5	Добавление клиента	Заполнить ФИО и телефон	Клиент появляется в таблице	Пройден
T6	Ошибка клиента	Оставить ФИО или телефон пустым	Появляется предупреждение	Пройден
T7	Создание заявки	Выбрать тур, клиента и количество туристов	Создается заявка, цена рассчитана автоматически	Пройден
T8	Недостаточно мест	Указать количество туристов больше остатка	Операция отменяется с сообщением	Пройден
T9	Добавление оплаты	Выбрать заявку и внести сумму	Платеж добавлен, статус обновлен	Пройден
T10	Формирование договора	Выбрать заявку и нажать «Сформировать»	В поле договора появляется текст	Пройден

4.2 Результаты тестирования

По результатам проверки основные сценарии работы модуля выполняются корректно. Пользователь может выполнить полный цикл: подобрать тур, зарегистрировать клиента, оформить заявку, внести оплату и сформировать договор.

Важным результатом является проверка бизнес-ограничений. Программа не позволяет создать заявку без выбранного тура или клиента, не принимает нулевую и отрицательную сумму оплаты, нормализует количество туристов до значения не меньше 1, а при недостатке мест возвращает понятное сообщение.

Сборка проекта была выполнена 21 мая 2026 года командой dotnet build. Результат: проект TourAgency1 успешно собран для net8.0, предупреждений 0, ошибок 0.

В ходе тестирования отдельное внимание уделялось согласованности финансовых данных. После добавления платежа сумма оплачено и остаток к оплате пересчитываются при повторной загрузке заявок. Это важно, потому

что итоговые значения выводятся не из временных переменных интерфейса, а из фактических записей базы данных.

Проверка интерфейса показала, что основные операции доступны без перехода по сложным меню. Пользователь видит таблицу туров, список клиентов, заявки, платежи и справочники на отдельных вкладках. Такой способ организации подходит для учебного приложения и сохраняет ясность пользовательского сценария.

4.3 Руководство пользователя

После запуска приложения на верхней панели можно указать направление, дату начала, максимальную цену и количество туристов. Кнопка «Подобрать» выводит подходящие туры, кнопка «Сбросить» возвращает полный список.

Для оформления заявки нужно выбрать тур, выбрать клиента, указать количество туристов и при необходимости примечание. После нажатия «Оформить заявку» программа рассчитывает стоимость и уменьшит количество свободных мест.

Если клиента еще нет в базе, его можно добавить на вкладке «Клиенты». Обязательными полями являются ФИО и телефон. Email и паспортные данные заполняются при наличии информации.

На вкладке «Заявки и оплаты» выбирается заявка, указывается сумма оплаты, способ оплаты и комментарий. После добавления оплаты статус заявки обновляется автоматически.

На вкладке «Договор» пользователь формирует текст договора по выбранной заявке. Команда «Сохранить и печатать» создает HTML-файл на рабочем столе в папке TourAgencyContracts и открывает его для печати.

При работе с программой рекомендуется после каждой операции обращать внимание на строку состояния под панелью поиска. В ней отображается результат загрузки данных, количество найденных туров, сообщение об успешном добавлении клиента, номер созданной заявки или описание ошибки.

Перед демонстрацией программы на защите следует заранее проверить доступность PostgreSQL и выполнить скрипты создания таблиц и тестовых данных. Если приложение запускается на другом компьютере, необходимо либо настроить такую же строку подключения, либо задать переменную окружения с адресом нужной базы.

Таблица 4.2 – Возможные затруднения пользователя

Ситуация	Действие
Данные не загрузились	Проверить доступность PostgreSQL, имя базы, логин, пароль и переменную окружения
Тур не найден	Уменьшить ограничения поиска или сбросить фильтры
Клиент не добавляется	Заполнить ФИО и телефон; проверить уникальность телефона
Заявка не создается	Выбрать тур и клиента; проверить количество свободных мест
Оплата не добавляется	Указать положительную сумму и выбрать заявку

4.4 Программа испытаний

Программа испытаний определяет порядок проверки готового программного модуля. Сначала проверяется техническая возможность запуска: восстановление пакетов, сборка решения и наличие исполняемого файла. Затем проверяется подключение к базе данных и загрузка демонстрационных записей.

После технической проверки выполняются пользовательские сценарии. Каждый сценарий имеет входные данные, последовательность действий и ожидаемый результат. Если фактический результат совпадает с ожидаемым, сценарий считается пройденным. Если результат отличается, фиксируется ошибка и выполняется исправление.

Для защиты курсовой работы достаточно продемонстрировать несколько ключевых сценариев: поиск тура, добавление клиента, оформление заявки, внесение оплаты и формирование договора. Эти операции показывают, что модуль работает с вводом, обработкой, хранением и выводом данных.

Таблица 4.3 – Критерии приемки программного модуля

Критерий	Условие	Подтверждение
Готовность к запуску	Проект собирается без ошибок	dotnet build завершен успешно

Готовность БД	Таблицы созданы, тестовые данные добавлены	В интерфейсе отображаются справочники
Корректность поиска	Фильтры ограничивают список туров	Найденные записи соответствуют условиям
Корректность заявки	Цена рассчитана автоматически	В bookings создана запись, в tours уменьшены места
Корректность оплаты	Платеж влияет на статус	Статус становится confirmed или paid
Корректность договора	Текст содержит данные выбранной заявки	Договор можно сохранить и распечатать

Ошибки подключения к базе данных не приводят к закрытию приложения. Пользователь видит текст ошибки в строке состояния. Это полезно при настройке учебного стенда, потому что позволяет отличить ошибку программы от недоступности PostgreSQL или неверной строки подключения.

Проверка с пустыми и некорректными значениями показала, что пользователь не может добавить клиента без обязательных полей, создать заявку без выбранного клиента или тура, а также внести оплату с нулевой суммой. Эти ограничения соответствуют требованиям к проверке корректности вводимых данных.

При работе с программой рекомендуется после каждой операции обращать внимание на строку состояния под панелью поиска. В ней отображается результат загрузки данных, количество найденных туров, сообщение об успешном добавлении клиента, номер созданной заявки или описание ошибки.

При печати договора рекомендуется сначала проверить сформированный текст на вкладке «Договор». Если данные клиента или заявки выбраны неверно, нужно вернуться к соответствующей вкладке, выбрать нужную запись и снова сформировать договор.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы был разработан программный модуль учета туров и заявок туристического агентства. Модуль реализован как настольное приложение на C# и .NET 8 с использованием Avalonia UI, ReactiveUI, PostgreSQL и Npgsql.

В работе выполнен анализ предметной области, сформулированы требования, спроектирована структура базы данных, разработан пользовательский интерфейс и реализованы основные бизнес-сценарии. Программа обеспечивает поиск туров, ведение клиентов, оформление заявок, учет оплат и формирование договора.

Разработанная база данных содержит связанные таблицы клиентов, направлений, отелей, туров, заявок и оплат. Ограничения целостности и транзакции повышают надежность операций. Проверка сборки подтвердила отсутствие ошибок компиляции.

Цель курсовой работы достигнута. Полученный программный модуль может использоваться как учебный пример разработки прикладного программного средства с графическим интерфейсом, базой данных и завершенным пользовательским сценарием.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.201-78. ЕСПД. Техническое задание. Требования к содержанию и оформлению.
2. ГОСТ 19.301-79. ЕСПД. Программа и методика испытаний. Требования к содержанию и оформлению.
3. ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления.
4. Павловская Т.А. С#. Программирование на языке высокого уровня. СПб.: Питер.
5. Шилдт Г. С# 4.0. Полное руководство. М.: Вильямс.
6. Рихтер Дж. CLR via С#. Программирование на платформе Microsoft .NET Framework. СПб.: Питер.
7. Microsoft Learn. Документация по языку С# и платформе .NET.
8. Microsoft Learn. Руководство по .NET 8.
9. Avalonia UI Documentation. Руководство по созданию кроссплатформенных интерфейсов.
10. ReactiveUI Documentation. Руководство по MVVM и реактивным командам.
11. Npgsql Documentation. Руководство по доступу к PostgreSQL из .NET.
12. PostgreSQL Documentation. Официальная документация по PostgreSQL.

ПРИЛОЖЕНИЯ

Приложение А. Фрагмент структуры базы данных

```
CREATE TABLE tours (  
    id integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    destination_id integer NOT NULL REFERENCES destinations (id) ON DELETE RESTRICT,  
    hotel_id integer NOT NULL REFERENCES hotels (id) ON DELETE RESTRICT,  
    title varchar(220) NOT NULL,  
    price_per_person numeric(12, 2) NOT NULL CHECK (price_per_person >= 0),  
    start_date date NOT NULL,  
    end_date date NOT NULL,  
    available_spots integer NOT NULL CHECK (available_spots >= 0),  
    meal_type varchar(80) NOT NULL DEFAULT 'Без питания',  
    transport_type varchar(80) NOT NULL DEFAULT 'Авиа',  
    CONSTRAINT tours_dates_check CHECK (end_date >= start_date)  
);
```

Приложение Б. Фрагмент модели заявки

```
public class Booking  
{  
    public int Id { get; set; }  
    public int TourId { get; set; }  
    public int ClientId { get; set; }  
    public string ClientName { get; set; } = string.Empty;  
    public string TourTitle { get; set; } = string.Empty;  
    public int PeopleCount { get; set; }  
    public decimal TotalPrice { get; set; }  
    public decimal PaidAmount { get; set; }  
    public decimal Balance => TotalPrice - PaidAmount;  
    public string Status { get; set; } = string.Empty;  
}
```

Приложение В. Фрагмент транзакции создания заявки

```
await using var connection = await dataSource.OpenConnectionAsync();  
await using var transaction = await connection.BeginTransactionAsync();  
  
await using var tourCommand = new NpgsqlCommand(tourSql, connection, transaction);  
tourCommand.Parameters.AddWithValue("tour_id", tourId);  
await using var reader = await tourCommand.ExecuteReaderAsync();  
  
if (!await reader.ReadAsync())  
{  
    throw new InvalidOperationException("Выбранный тур не найден.");  
}  
  
var price = reader.GetDecimal(0);  
var availableSpots = reader.GetInt32(1);  
await reader.CloseAsync();  
  
if (availableSpots < peopleCount)  
{  
    throw new InvalidOperationException($"Недостаточно мест: доступно {availableSpots}.");  
}
```


Приложение Г. Экранные формы программного модуля

The screenshot shows a web application window titled 'Туристическое агентство'. The header includes the agency name and a subtitle 'Подбор путевок, оформление заявок, оплаты и договоры', along with an 'Обновить' button. The main search area contains input fields for 'Направление, страна или город' (with the example 'например, Сочи'), 'Дата начала от' (01.06.2026), 'Цена до' (50000), and 'Туристов' (1). There are 'Подобрать' and 'Сбросить' buttons. Below this, a status message reads 'Готово: направлений 4, отелей 4, туров 4, заявок 3.' A navigation bar with tabs 'Путевки', 'Клиенты', 'Заявки и оплаты', 'Договор', and 'Справочники' is present. The central part of the interface is a table listing four travel options. The bottom section contains a summary area with fields for 'Клиент для заявки' (Иванов Иван Петрович), 'Выбранная путевка' (Сочи, Россия, Морской бриз, 01.06.2026), 'Туристов' (1), and 'Примечание к заявке' (пожелания клиента), with an 'Оформить заявку' button.

№	Направление	Отель	Тур	Питание	Транспорт	Цена за чел.	Начало	Окончание	
1	Сочи, Россия	Морской бриз	4	Черноморская неделя	Завтраки	Ж/д	25 000,00	01.06.2026	08.06.2026
2	Стамбул, Турция	Golden Horn View	4	Стамбул на двоих берегах	Завтраки	Авиа	35 000,00	10.06.2026	17.06.2026
3	Ереван, Армения	Cascade Boutique	5	Вкус Армении	Полупансис	Авиа	42 000,00	05.07.2026	12.07.2026
4	Калининград, Россия	Балтика Парк	3	Балтийские каникулы	Завтраки	Автобус	30 000,00	03.08.2026	09.08.2026

Рисунок Г.1 – Экранная форма подбора путевок

Туристическое агентство

Подбор путевок, оформление заявок, оплаты и договоры

Обновить

Направление, страна или город

Дата начала от

Цена до

Туристов

Подобрать

Сбросить

например, Сочи

01.06.2026

50000

1

Готово: направлений 4, отелей 4, туров 4, заявок 3.

Путевки

Клиенты

Заявки и оплаты

Договор

Справочники

№	ФИО	Телефон	Email	Паспорт
1	Иванов Иван Петрович	+79001234567	ivanov@example.com	4512 123456
2	Петрова Анна Сергеевна	+79007654321	petrova@example.com	4513 654321
3	Смирнов Олег Андреевич	+79005553322	smirnov@example.com	4514 112233

Новый клиент

ФИО

Телефон

Email

Паспорт

Добавить клиента

Рисунок Г.2 – Экранная форма работы с клиентами

Туристическое агентство

Туристическое агентство

Подбор путевок, оформление заявок, оплаты и договоры

Обновить

Направление, страна или город

например, Сочи

Дата начала от

01.06.2026

Цена до

50000

Туристов

1

Подобрать

Сбросить

Готово: направлений 4, отелей 4, туров 4, заявок 3.

Путевки

Клиенты

Заявки и оплаты

Договор

Справочники

№	Клиент	Направление	Отель	Турист	Стоимость	Оплачено	Остаток	Статус	Создана
3	Смирнов Олег Андреевич	Ереван, Армения	Cascade Boutique	3	126 000,00	0,00	126 000,00	new	17.05.2026 09:1
2	Петрова Анна Сергеевна	Стамбул, Турция	Golden Horn View	1	35 000,00	35 000,00	0,00	paid	16.05.2026 12:5
1	Иванов Иван Петрович	Сочи, Россия	Морской бриз	2	50 000,00	15 000,00	35 000,00	confirmed	15.05.2026 10:0

Сумма

126000

Способ

Карта

Комментарий

Добавить оплату

ID	Заявка	Дата	Сумма	Способ	Комментарий
2	2	16.05.2026 13:00	35 000,00	Перевод	Полная оплата
1	1	15.05.2026 10:30	15 000,00	Карта	Предоплата

Рисунок Г.3 – Экранная форма учета заявок и оплат

Туристическое агентство

Подбор путевок, оформление заявок, оплаты и договоры

Обновить

Направление, страна или город: например, Сочи

Дата начала от: 01.06.2026

Цена до: 50000

Туристов: 1

Подобрать Сбросить

Готово: направлений 4, отелей 4, туров 4, заявок 3.

Путевки Клиенты Заявки и оплаты Договор Справочники

Сформировать Сохранить и печатать

ДОГОВОР №3 НА РЕАЛИЗАЦИЮ ТУРИСТСКОГО ПРОДУКТА

Дата формирования: 21.05.2026

Туристическое агентство оформляет для клиента Смирнов Олег Андреевич паспорт: 4514 112233, телефон: +79005553322 путевку по направлению: Ереван, Армения.

Тур: Вкус Армении
Отель: Cascade Boutique
Даты поездки: 05.07.2026 - 12.07.2026
Количество туристов: 3

Полная стоимость тура: 126 000,00 руб.
Оплачено: 0,00 руб.
Остаток к оплате: 126 000,00 руб.
Статус оформления: Новая

Примечания: Семейное размещение

Подписи сторон:
Агент _____
Клиент _____

Рисунок Г.4 – Экранная форма формирования договора

Туристическое агентство

Туристическое агентство

Подбор путевок, оформление заявок, оплаты и договоры

Обновить

Направление, страна или город

например, Сочи

Дата начала от

01.06.2026

Цена до

50000

Туристов

1

Подобрать

Сбросить

Готово: направлений 4, отелей 4, туров 4, заявок 3.

Путевки

Клиенты

Заявки и оплаты

Договор

Справочники

№	Страна	Город	Направление	Описание
3	Армения	Ереван	Ереван, Армения	Гастрономич
4	Россия	Калининград	Калининград, Россия	Балтийские к
1	Россия	Сочи	Сочи, Россия	Пляжный отд
2	Турция	Стамбул	Стамбул, Турция	Экскурсионн

№	Направление	Отель		Адрес
3	Ереван, Армения	Cascade Boutique	5	Ереван, ул. Абовяна, 8
4	Калининград, Росс	Балтика Парк	3	Калининград, ул. Озер
1	Сочи, Россия	Морской бриз	4	Сочи, Курортный прос
2	Стамбул, Турция	Golden Horn View	4	Istanbul, Fatih, 22

Рисунок Г.5 – Экранная форма справочников направлений и отелей